

3D Artillery: Castle Siege

Project Report

Asaf Kanlıpıçak (2020400195)

CMPE 485 Introduction to Game Programming

Repo: <https://github.com/Asaf31214/cmpe-485/tree/main/term-project>

Demo: https://drive.google.com/file/d/1lB9lTUCHqQcztY_33ciYmvXIWBL4yNsM/view?usp=sharing

1. Introduction

This project is a physics-based 3D artillery game built in Unity. The player controls a cannon with the goal of destroying all red target blocks inside stacked-block castles using a limited number of cannonballs. The game was developed under Option B (Simple Game) and uses only Unity's built-in primitives and PhysX; all scene objects, including the cannon, castles, lighting, and UI, are created at runtime through code with no editor prefabs or external frameworks.

2. Game Mechanics & User Experience

The core loop consists of aiming, firing, and resolving. The player adjusts the cannon's yaw and pitch with the arrow keys, changes shot power with C and V, and fires with Space. A trajectory preview made of parabolic dot markers helps the player estimate the shot path before committing.

Three block types define the levels:

- **Block:** Standard grey physics block that can be damaged and pushed by explosions.
- **Target:** Red cylindrical objective. It must be destroyed to win the level.
- **Anchor:** Static blue block with no Rigidbody, used to create floating or suspended structures. Can also be destroyed by explosion damage.

The game contains eight levels. Levels 1–5 are normal gameplay scenarios of increasing architectural complexity, from simple stacks to arches, towers, and fortresses. Levels 6–8 are controlled performance-test scenes containing approximately 1,000, ~5,000, and ~10,000 blocks respectively. In these levels, the player can press N to drop a high-radius "nuke" projectile and O to toggle between standard and optimized explosion processing.

Win condition: destroy all targets before running out of ammo. Lose condition: ammo reaches zero with targets still standing. After each shot, the game waits for physics to settle before allowing the next turn.

3. Software Design & Architecture

The project follows a runtime-only philosophy. Every object is instantiated and configured in code via a central entry script that sets up physics settings, lighting, the ground plane, the cannon, and the GameManager. This removes dependency on Unity prefabs and keeps the entire scene definition version-controllable.

GameManager acts as a singleton and drives a finite-state machine with four states: Aim, Fire, Resolve, and GameOver. Cross-object communication is handled through C# events (for example, OnFire and OnTargetDestroyed), which decouples the cannon, projectiles, and UI from the manager.

Key subsystems include:

- **CannonController:** Handles input and procedurally builds the cannon mesh from primitives.
- **Projectile:** A Rigidbody-driven shell that explodes on impact or when it falls below the world bounds.
- **ExplosionManager:** A static utility that handles area-of-effect logic.
- **CastleBuilder:** Reads level layouts from 3D integer arrays and constructs castles block by block. It also maintains a cached list of all destructible blocks in optimized mode, which becomes important for performance scaling.

LevelData stores all level configurations as static 3D array layouts, ammo counts, and power settings. The GameUI canvas displays ammo, remaining targets, current level, power level, and a lightweight profiler overlay showing FPS, physics object count, and memory allocation.

4. Technical Challenges

Three main challenges emerged during development:

Physics Stability on Stacked Blocks

When blocks were stacked tightly, they jittered or collapsed before the player fired. This was addressed by tuning global physics settings (lowering sleepThreshold, raising solverIterations, using ContinuousSpeculative collision detection, and adding small amounts of linear and angular drag) rather than switching blocks to kinematic mode, which would have suppressed the desired explosion-driven chaos.

Radial Explosion Logic

Explosions need to apply both force and damage to every object within a radius. The solution uses `Physics.OverlapSphere` to gather colliders, then applies `AddExplosionForce` to their `Rigidbody`s and computes distance-based damage falloff. A simpler manual raycast approach was considered but rejected as more expensive for dense scenes.

Game State Synchronization

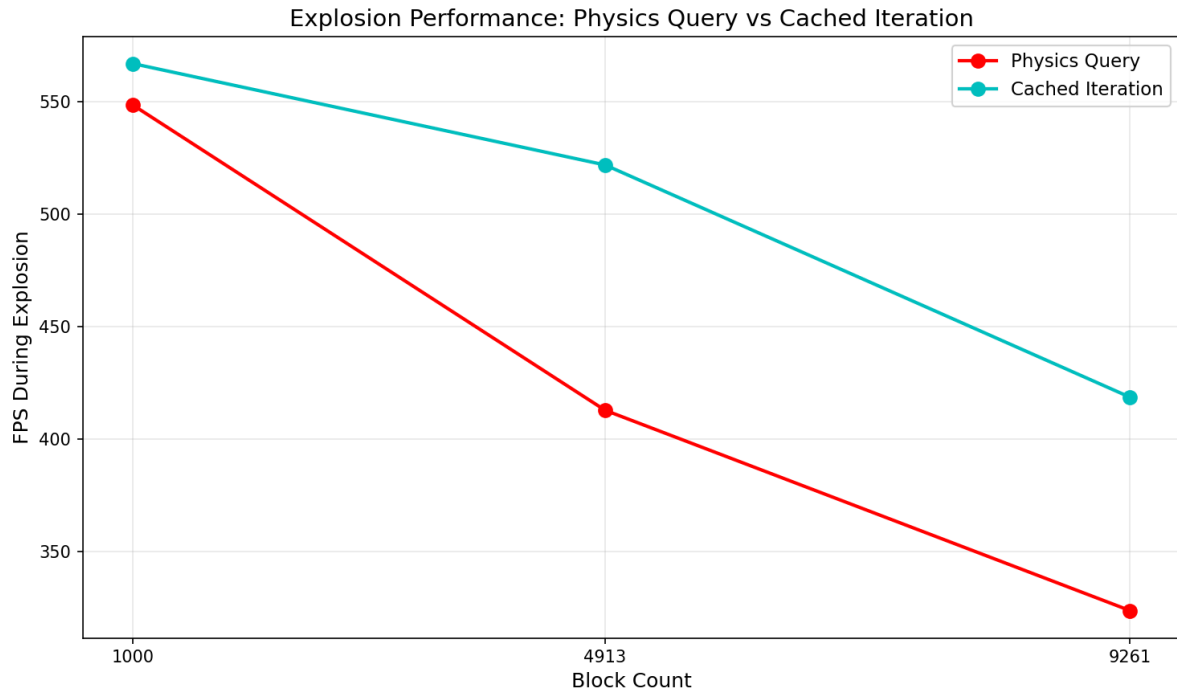
Because physics resolution is asynchronous, the FSM cannot instantly transition from Fire to Aim. A coroutine-based sequence in `GameManager` waits for all projectiles to fall below a velocity threshold (with a safety timeout), then transitions to Resolve, checks win/lose conditions, and finally returns to Aim or ends the game.

5. Performance Evaluation

The technically demanding part of the project is the large-scale explosion processing in levels 6-8. Two approaches were compared:

- 1. Physics Query:** Using `Physics.OverlapSphere` to find colliders inside the blast radius, then retrieving the `Destructible` component from each hit.
- 2. Cached Iteration:** Iterating the pre-cached list of blocks from `CastleBuilder` and performing a simple distance check, skipping the physics query entirely.

Metrics were captured with a custom `PerformanceLogger` using Stopwatch timing and FPS sampling during the explosion frame. Each test was run in a controlled environment by loading a nuke level, pressing N, and recording the results.



The results show that both methods perform similarly at 1,000 blocks, but the cached iteration approach scales noticeably better as block count grows, maintaining higher FPS and lower processing time at ~5,000 and ~10,000 blocks. The improvement comes from eliminating the overhead of the physics spatial query and repeated component lookups at scale.

6. Conclusion

The project demonstrates that a complete physics-driven artillery game can be built entirely from runtime code with clean architectural separation via an event-driven FSM. The main takeaway is that caching game-object references and bypassing repeated physics queries becomes a meaningful optimization when destructive interactions scale to thousands of rigid bodies. Future extensions could include more complex castle architectures, additional projectile types, or a spline-based cinematic camera path during the fire sequence.